QueryAPIGate HOW-TO GUIDE 44 · EVENTS & EXPORTS

Deliver a daily file to a partner's bucket

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=44-deliver-a-daily-file-to-a-partners-bucket

---

Time: 20 minutes. **You'll end up with:** a saved query delivered every day as a Parquet file to a bucket your partner reads from - only the rows that are new since the last delivery, with every run on record, an alert when one fails, and nothing ever written outside the prefix you allowed.

Every command and response below was run against a real server and an S3-compatible bucket (SeaweedFS), with the example APIs loaded. `$TOKEN` is an owner's or admin's admin token (the shared key works too).

Step 1: A query that can say "only what's new"

An incremental export remembers the largest value of one column it has delivered - the **watermark** - and binds it to a parameter on the next run. So the query needs a column that only grows, and a parameter to compare it with:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/queries -H "X-API-Key: $TOKEN" -H 'Content-Type:
application/json' -d '{
  "name": "rentals_after", "connection_name": "examples", "description": "Rentals after an id",
"publish": true,
  "sql": "SELECT rental_id, rental_date, amount FROM rental WHERE rental_id > :after ORDER BY
rental_id",
  "parameters": {"after": {"type": "int", "min": 0}}
}'
```

Pick the column with care. An id that only grows (`rental_id`) is the safest: nothing is missed and nothing is sent twice. A timestamp (`updated_at > :since`) works too, but `>` can miss a row written later with the same timestamp as the last one delivered, and `≥` re-sends the rows on the boundary. And the parameter must accept what the column holds: a date-only parameter can't take a timestamp watermark - the run says so, and writes nothing.

Step 2: The destination - where it may write, and nowhere else

```shell
curl -X POST http://127.0.0.1:5000/api/v1/destinations -H "X-API-Key: $TOKEN" -H 'Content-Type:
application/json' -d '{
  "name": "acme", "url": "s3://acme-exchange/from-us/",
  "user": "AKIA...", "password": "${ACME_SECRET_KEY}", "region": "eu-west-1"
}'
```

- `url` is the prefix exports may write under: `s3://` , `gs://` (with `"storage": "gcs"` HMAC keys), `r2://` (with `account_id`), or a local folder - which must sit inside one the operator set in `QUERYAPIGATE_EXPORT_DIRS` , never the server's own home. For an S3-compatible service, add `endpoint` , `"url_style": "path"` and `"use_ssl": false` without TLS.

- The secret is stored like a connection password: here a `${ACME_SECRET_KEY}` reference to the server's environment; a typed one is masked in every response and encrypted with `QUERYAPIGATE_SECRET_KEY`.
- **Nothing can be written outside the prefix.** Every write goes through a DuckDB connection locked to it - no path, parameter or file name can reach `s3://acme-exchange/other-partner/`; DuckDB refuses. Still give the credentials write access to that prefix only: two locks.

Test it before anything depends on it - it writes one small file, `_queryapigate_probe.csv`, overwritten each time:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/destinations/acme/test -H "X-API-Key: $TOKEN"
# {"object": "s3://acme-exchange/from-us/_queryapigate_probe.csv", "elapsed_ms": 251.0}
```

In the Console: **Delivery** → **Destinations** → **New destination**, with a **Test** button in the form.

Step 3: The export

```shell
curl -X POST http://127.0.0.1:5000/api/v1/exports -H "X-API-Key: $TOKEN" -H 'Content-Type:
application/json' -d '{
  "name": "acme-rentals", "query": "rentals_after", "destination": "acme",
  "path": "rentals/{date}/rentals_{run}.parquet",
  "incremental": {"column": "rental_id", "parameter": "after", "start": 0}
}'
```

- `path` is under the destination. `{date}` (`YYYY-MM-DD`), `{time}`, `{run}` (the run's id - every file unique), `{name}` and the query's parameters (`{region}`, plain values only) are filled in.
- `format`: `parquet` (default), `csv` or `ndjson`, written by DuckDB with the rows in the query's order. A Parquet file keeps the column types - decimals exact, dates as dates.
- It's checked now, not at 3 a.m.: the destination and the published query must exist, the query must use `:after`, every placeholder must fill in.

Step 4: Run it

```shell
curl -X POST http://127.0.0.1:5000/api/v1/exports/acme-rentals/runs -H "X-API-Key: $TOKEN"
```

```json
{"id": "ec7e7e7e8cb5", "export": "acme-rentals", "rows": 20000,
 "object": "s3://acme-exchange/from-us/rentals/2026-10-09/rentals_ec7e7e7e8cb5.parquet",
 "watermark": {"from": null, "to": 20000}, "duration_ms": 100.7}
```

Run again straight away, and there is nothing new - so no file (`skip_empty`, on by default):

```json
{"id": "3ef429ef0f03", "rows": 0, "object": null, "watermark": {"from": 20000, "to": 20000}}
```

Three new rentals later, the next run delivers exactly those:

```json
{"id": "622ec9e44786", "rows": 3,
 "object": "s3://acme-exchange/from-us/rentals/2026-10-09/rentals_622ec9e44786.parquet",
 "watermark": {"from": 20000, "to": 20003}}
```

In the Console: **Delivery** → **Exports** → **Run now**, and **Runs** for its history.

Step 5: Put it on a schedule

QueryAPIGate doesn't keep a clock; your scheduler calls it. From the server's own host - no server needed, it reads the store directly:

```shell
# crontab: every day at 06:00
0 6 * * *  QUERYAPIGATE_HOME=/srv/queryapigate ACME_SECRET_KEY=... queryapigate exports run acme-rentals
# Wrote 3 rows to s3://acme-exchange/from-us/rentals/2026-10-09/rentals_622ec9e44786.parquet (run
622ec9e44786).
# Watermark: 20000 → 20003
```

Or over HTTP, from a Kubernetes CronJob or Airflow, with a token of its own - a `developer` may run exports, but not define or change them:

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: acme-rentals
spec:
  schedule: "0 6 * * *"
  concurrencyPolicy: Forbid
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: run
              image: curlimages/curl:8.10.1
              args: ["-fsS", "-X", "POST", "-H", "X-API-Key: $(QUERYAPIGATE_TOKEN)",
                     "http://queryapigate:5000/api/v1/exports/acme-rentals/runs"]
              env:
                - name: QUERYAPIGATE_TOKEN
                  valueFrom: {secretKeyRef: {name: queryapigate-ci, key: token}}
```

`-f` makes a failed run fail the job, so Kubernetes retries it and reports it. Two runs of one export never overlap, whoever starts them: the second answers `409 export_running` (with `Retry-After`) until the first finishes - and a run that crashed holds the export only until its lease expires.

When a run fails

With the bucket unreachable:

```json
{"error": "Couldn't write s3://acme-exchange/from-us/rentals/2026-10-09/rentals_a55563feae35.parquet: IO Error:
 Could not connect to server ...", "code": "destination_unreachable"}
```

- **Nothing is lost.** The watermark moves only after a file is written, so it stayed at `20003`. When the bucket was back, the next run delivered the one waiting row - `{"rows": 1, "watermark": {"from": 20003, "to": 20004}}`. An object store upload is all or nothing: a failed run leaves no partial file.
- **You're told.** The `export_failing` alert - *Export 'acme-rentals' is failing* - is on the Alerts screen and `GET /api/v1/alerts` until a run succeeds, and every run, failed or not, is in `GET /api/v1/exports/acme-rentals/runs` and run history, with who ran it.
- **Re-send on purpose** by moving the watermark back - audited: `PATCH /api/v1/exports/acme-rentals` with `{"watermark": 19000}`, or `{"watermark": null}` to start again from `start`.

Good to know

- **Memory.** A run streams the rows through the server once, into a temporary file, and DuckDB writes from it: about 150 MB for a local file whatever the size, and 180-300 MB to S3 for 2-6 million rows (measured; the S3 upload's parts grow a little with the file).
- **Who may do what.** Owners and admins define destinations and exports - that decides who receives data. Developers may run them; auditors may read them. Scoped API keys can't touch them.
- **What's in the files** is what the query returns. To hide columns from a partner - email addresses, say - give the export masking rules.
- **One-off files**, without saving an export: `queryapigate export rentals_after --param after=0 --to acme --format csv --out 'adhoc/{date}.csv'`.

Next steps

- Run a scheduled export to a local file - the same, without a destination.
- Give your team their own admin access - a token for the CronJob.
- API reference: exports to object storage.